

```
# movie_predictor_streamlit.py - Movie Success Prediction with Streamlit
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
import pickle
import streamlit as st
import plotly.express as px
import plotly.graph_objects as go
```

```
# 1. CREATE SAMPLE DATASET
```

```
@st.cache_data
```

```
def create_movie_dataset():
```

```
    """Create a sample movie dataset"""
```

```
    np.random.seed(42)
```

```
    n_samples = 100
```

```
    data = {
```

```
        'budget': np.random.uniform(1, 200, n_samples),
```

```
        'runtime': np.random.uniform(80, 180, n_samples),
```

```
        'rating': np.random.choice(['G', 'PG', 'PG-13', 'R'], n_samples),
```

```
        'genre': np.random.choice(['Action', 'Comedy', 'Drama', 'Horror', 'Romance'], n_samples),
```

```
        'director_experience': np.random.randint(1, 30, n_samples),
```

```
        'star_power': np.random.uniform(1, 10, n_samples),
```

```
        'marketing_spend': np.random.uniform(5, 100, n_samples),
```

```
        'release_month': np.random.randint(1, 13, n_samples),
```

```
        'sequel': np.random.choice([0, 1], n_samples, p=[0.7, 0.3]),
```

```
    }
```

```
    success_prob = (
```

```
        (data['budget'] / 200) * 0.2 +
```

```
        (data['star_power'] / 10) * 0.3 +
```

```
        (data['marketing_spend'] / 100) * 0.2 +
```

```
        (data['sequel']) * 0.2 +
```

```
        np.random.uniform(0, 0.1, n_samples)
```

```
    )
```

```
    data['success'] = ['Hit' if prob > 0.5 else 'Flop' for prob in success_prob]
```

```
    df = pd.DataFrame(data)
```

```
    df.to_csv('movie_dataset.csv', index=False)
```

```
    return df
```

```
# 2. MOVIE SUCCESS PREDICTOR CLASS
```

```
class MovieSuccessPredictor:
```

```
    def __init__(self):
```

```
        self.model = None
```

```
        self.encoders = {}
```

```
        self.feature_columns = None
```

```
    def load_data(self, filepath='movie_dataset.csv'):
```

```
        """Load dataset"""
```

```
        try:
```

```

df = pd.read_csv(filepath)
return df
except FileNotFoundError:
return create_movie_dataset()

def preprocess_data(self, df):
"""Preprocess the data"""
df_processed = df.copy()
categorical_cols = ['rating', 'genre']

for col in categorical_cols:
le = LabelEncoder()
df_processed[col] = le.fit_transform(df_processed[col])
self.encoders[col] = le

X = df_processed.drop('success', axis=1)
y = df_processed['success']
self.feature_columns = X.columns.tolist()

return X, y

def train_model(self, X, y):
"""Train Decision Tree"""
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)

self.model = DecisionTreeClassifier(
random_state=42, max_depth=10,
min_samples_split=5, min_samples_leaf=3
)

self.model.fit(X_train, y_train)
y_pred = self.model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

return accuracy, y_test, y_pred, X_train, X_test

def save_model(self, filepath='movie_predictor.pkl'):
"""Save model"""
model_data = {
'model': self.model,
'encoders': self.encoders,
'feature_columns': self.feature_columns
}
with open(filepath, 'wb') as f:
pickle.dump(model_data, f)

def load_model(self, filepath='movie_predictor.pkl'):
"""Load model"""
try:
with open(filepath, 'rb') as f:
model_data = pickle.load(f)
self.model = model_data['model']
self.encoders = model_data['encoders']
self.feature_columns = model_data['feature_columns']
return True

```

```
except FileNotFoundError:
    return False
```

```
def predict(self, movie_params):
    """Predict movie success"""
    if self.model is None:
        return None, None
```

```
df = pd.DataFrame([movie_params])
```

```
for col in ['rating', 'genre']:
    if col in df.columns and col in self.encoders:
        df[col] = self.encoders[col].transform(df[col])
```

```
prediction = self.model.predict(df)[0]
probability = self.model.predict_proba(df)[0].max()
```

```
return prediction, probability
```

```
# Initialize predictor
@st.cache_resource
def get_predictor():
    predictor = MovieSuccessPredictor()
    if not predictor.load_model():
        # Train new model
        df = predictor.load_data()
        X, y = predictor.preprocess_data(df)
        accuracy, _, _, _ = predictor.train_model(X, y)
        predictor.save_model()
        st.session_state.model_accuracy = accuracy
        st.session_state.dataset = df
    return predictor
```

```
# STREAMLIT APP
def main():
    st.set_page_config(
        page_title=" Movie Success Predictor",
        layout="wide",
        initial_sidebar_state="expanded"
    )
```

```
st.title("Movie Success Predictor")
st.markdown("### Predict whether a movie will be a Hit or Flop using AI")
```

```
# Initialize predictor
predictor = get_predictor()
```

```
# Sidebar for model info
with st.sidebar:
    st.header(" Model Information")
```

```
if 'model_accuracy' not in st.session_state:
    # Load dataset for info
    df = predictor.load_data()
    st.session_state.dataset = df
    st.session_state.model_accuracy = 0.85 # Default value
```

```
st.metric("Model Accuracy", f"{st.session_state.model_accuracy:.1%}")
st.metric("Dataset Size", len(st.session_state.dataset))
```

```
# Dataset distribution
success_counts = st.session_state.dataset['success'].value_counts()
fig_pie = px.pie(
    values=success_counts.values,
    names=success_counts.index,
    title="Success Distribution",
    color_discrete_map={'Hit': '#00CC96', 'Flop': '#EF553B'}
)
fig_pie.update_layout(height=300)
st.plotly_chart(fig_pie, use_container_width=True)
```

```
if st.button("Retrain Model"):
    df = create_movie_dataset()
    X, y = predictor.preprocess_data(df)
    accuracy, _, _, _ = predictor.train_model(X, y)
    predictor.save_model()
    st.session_state.model_accuracy = accuracy
    st.session_state.dataset = df
    st.success(f"Model retrained! New accuracy: {accuracy:.1%}")
    st.experimental_rerun()
```

```
# Main content area
col1, col2 = st.columns([1, 1])
```

```
with col1:
    st.subheader(" Movie Parameters")
```

```
# Input form
with st.form("movie_form"):
    budget = st.slider("Budget (millions USD)", 1, 500, 50)
    runtime = st.slider(" Runtime (minutes)", 60, 300, 120)
```

```
col_a, col_b = st.columns(2)
with col_a:
    rating = st.selectbox(" Rating", ['G', 'PG', 'PG-13', 'R'], index=2)
    genre = st.selectbox("Genre", ['Action', 'Comedy', 'Drama', 'Horror', 'Romance'])
```

```
with col_b:
    director_exp = st.slider("    Director Experience (years)", 1, 50, 10)
    star_power = st.slider("★ Star Power (1-10)", 1.0, 10.0, 7.0, 0.1)
```

```
marketing = st.slider(" Marketing Spend (millions USD)", 1, 200, 30)
```

```
col_c, col_d = st.columns(2)
with col_c:
    release_month = st.selectbox("Release Month",
    list(range(1, 13)),
    format_func=lambda x: f"Month {x}",
    index=5)
with col_d:
    sequel = st.selectbox(" Is Sequel?", [0, 1],
    format_func=lambda x: "Yes" if x else "No")
```

```
predict_button = st.form_submit_button("Predict Success",
use_container_width=True)
```

```
with col2:
st.subheader(" Prediction Results")
```

```
if predict_button:
movie_params = {
'budget': float(budget),
'runtime': float(runtime),
'rating': rating,
'genre': genre,
'director_experience': int(director_exp),
'star_power': float(star_power),
'marketing_spend': float(marketing),
'release_month': int(release_month),
'sequel': int(sequel)
}
```

```
prediction, confidence = predictor.predict(movie_params)
```

```
if prediction:
# Display prediction
if prediction == 'Hit':
st.success(f" **{prediction}**")
st.balloons()
else:
st.error(f"***{prediction}**")
```

```
# Confidence metric
st.metric("Confidence", f"{confidence:.1%}")
```

```
# Confidence gauge
fig_gauge = go.Figure(go.Indicator(
mode = "gauge+number+delta",
value = confidence * 100,
domain = {'x': [0, 1], 'y': [0, 1]},
title = {'text': "Confidence Level"},
gauge = {
'axis': {'range': [None, 100]},
'bar': {'color': "darkblue"},
'steps': [
{'range': [0, 50], 'color': "lightgray"},
{'range': [50, 80], 'color': "yellow"},
{'range': [80, 100], 'color': "green"}
],
'threshold': {
'line': {'color': "red", 'width': 4},
'thickness': 0.75,
'value': 90
}
}
))
fig_gauge.update_layout(height=300)
st.plotly_chart(fig_gauge, use_container_width=True)
```

```
# Movie summary
```

```

st.subheader(" Movie Summary")
st.write(f"""
**Genre:** {genre} | **Rating:** {rating} | **Runtime:** {runtime} min

**Budget:** ${budget}M | **Marketing:** ${marketing}M

**Director Experience:** {director_exp} years | **Star Power:** {star_power}/10

**Release:** Month {release_month} | **Sequel:** {'Yes' if sequel else 'No'}
""")

```

```

else:
st.info(" Fill out the form and click 'Predict Success' to see results!")

```

```

# Feature importance chart
if predictor.model:
feature_names = predictor.feature_columns
importances = predictor.model.feature_importances_

fig_importance = px.bar(
x=importances,
y=feature_names,
orientation='h',
title=" Feature Importance",
labels={'x': 'Importance', 'y': 'Features'}
)
fig_importance.update_layout(height=400)
st.plotly_chart(fig_importance, use_container_width=True)

```

```

# Dataset viewer
with st.expander(" View Training Dataset"):
st.dataframe(st.session_state.dataset, use_container_width=True)

```

```

# Quick examples
st.subheader(" Quick Examples")

```

```

col1, col2, col3 = st.columns(3)

```

```

with col1:
if st.button(" Superhero Blockbuster", use_container_width=True):
st.session_state.update({
'budget': 200, 'runtime': 150, 'rating': 'PG-13', 'genre': 'Action',
'director_exp': 15, 'star_power': 9.0, 'marketing': 100,
'release_month': 5, 'sequel': 1
})
st.experimental_rerun()

```

```

with col2:
if st.button(" Indie Drama", use_container_width=True):
st.session_state.update({
'budget': 15, 'runtime': 105, 'rating': 'R', 'genre': 'Drama',
'director_exp': 8, 'star_power': 6.0, 'marketing': 5,
'release_month': 10, 'sequel': 0
})
st.experimental_rerun()

```

```

with col3:

```

```
if st.button(" Horror Flick", use_container_width=True):
    st.session_state.update({
        'budget': 25, 'runtime': 90, 'rating': 'R', 'genre': 'Horror',
        'director_exp': 5, 'star_power': 4.0, 'marketing': 15,
        'release_month': 10, 'sequel': 0
    })
    st.experimental_rerun()
```

```
if __name__ == "__main__":
    main()
```